

longops

Long operations and detailed calculations.

Opérations posées et calculs détaillés.

v0.2.0

Akilon <<https://github.com/Akilon27>>

MIT

A propos

En tant que professeur de mathématiques, j'ai parfois besoin de montrer comment poser un calcul « à la main ». Je mets maintenant à disposition des fonctions que j'ai créées initialement pour moi. J'utilisais précédemment le paquet LaTeX « xlop » et, typst étant beaucoup plus commode d'accès, j'ai voulu en faire un équivalent moi-même. Ce paquet n'est pas une traduction directe de « xlop », il a des fonctionnalités en plus et d'autres en moins, selon mes besoins.

J'ai parfois utilisé des IA (principalement Gemini pour la partie « Priorités » ou pour ajouter aux autres fonctions des possibilités que j'avais d'abord codées moi-même pour l'une d'entre elles).

Remerciements : Je tiens à remercier [ObaulG](#), la partie « durées/horaires » vient en grande partie de son travail.

Fonctions pour les calculs posés « à la main » ...

- `add-en-ligne()`
- `addition()`
- `base-to-base()`
- `division()`
- `egalite-decimale()`
- `egalite-euclidienne()`
- `from-base()`
- `multiplication()`
- `racine()`
- `sous-en-ligne()`
- `soustraction()`
- `to-base()`

add-en-ligne

Pour avoir la somme en ligne de tous les nombres saisis

```
$#add-en-ligne(198.7,120,35)$
```

```
198.7 + 120 + 35 = 353.7
```

Paramètres

```
add-en-ligne(..ops: arguments) -> content
```

..ops arguments

termes de l'addition

addition

Pose l'addition de 2 ou plusieurs nombres avec possibilité de cacher la ligne de résultat avec `hide-result`, une list de certains chiffres puis d'afficher la solution, diverses couleurs paramétrables etc

Exemples :

```
#addition(12.5, 3.75, .8, 14.2) #h(1cm)
#addition(12.5, 3.75, .8, 14.2, list:
(2,10,19)) #h(1cm)
#addition(base: 16, "1A.F", "2B.8", show-
base: true,)
```

$\begin{array}{r} 12,50 \\ + 3,75 \\ + 0,80 \\ + 14,20 \\ \hline 31,25 \end{array}$	$\begin{array}{r} 1\bigcirc,50 \\ + 3,\bigcirc5 \\ + 0,80 \\ + \bigcirc4,20 \\ \hline 31,25 \end{array}$	$\begin{array}{r} 11 \\ 1A,F \\ + 2B,8 \\ \hline 46,7^{16} \end{array}$
---	--	---

Paramètres

```
addition(  
  base: int ,  
  show-base: boolean ,  
  show-carry: boolean ,  
  carries-color: color ,  
  size: length relative length ,  
  fr: boolean ,  
  hide-result: boolean ,  
  border-color: color ,  
  solution-color: color ,  
  type-mask: str ,  
  list: array ,  
  solution: boolean ,  
  sign: boolean content ,  
  space: length relative length fraction ,  
  text-size: length relative length ,  
  ..args: array arguments  
) -> content
```

base `int`

Base numérique (de 2 à 36)

Par défaut: `10`

show-base `boolean`

Montrer ou non dans quelle base l'addition est posée

Par défaut: `false`

show-carry `boolean`

Montrer ou non les retenues

Par défaut: `true`

carries-color `color`

couleur des retenues

Par défaut: `red`

size `length` or `relative length`

Taille des colonnes

Par défaut: `1em`

fr `boolean`

Affichage des nombres à la française « , » ou pas

Par défaut: `true`

hide-result `boolean`

Cache les retenues et le résultat

Par défaut: `false`

border-color `color`

Couleur du cadre des chiffres cachés

Par défaut: `blue.mix(gray)`

solution-color `color`

Couleur des chiffres cachés si solution = true

Par défaut: `red`

type-mask `str`

Comment les chiffres sont cachés : soient « rect » pour un rectangle arrondi soit ...

Par défaut: `"rect"`

list `array`

list des chiffres à cacher dans une addition à trous

Par défaut: `()`

solution `boolean`

Montrer les retenues et le résultat en mode hide-result (pour les corrections)

Par défaut: `false`

sign `boolean` or `content`

sign d'égalité, false ou true [=] ou un symbole \$ = \$

Par défaut: `false`

space length or relative length or fraction

Espacement si plusieurs sur la même ligne

Par défaut: 1fr

text-size length or relative length

taille du des chiffres : 1em

Par défaut: 1em

..args array or arguments

termes à additionner

base-to-base

conversion d'un nombre d'une base (défaut : 2) vers une autre base (défaut : 10).

```
// Conversion entre deux bases non  
décimales (Hexadécimal vers Binaire) :  
#base-to-base("1A", base-from: 16, base-  
to: 2, equal: true)  
// Conversion octal vers binaire :  
#base-to-base("77", base-from: 8, base-  
to: 2, equal: true, mode:"line")
```

$$(1A)_{16} = (11010)_2$$

$$\overline{77}^8 = \overline{111111}^2$$

```
// Explication complète pas à pas :  
#base-to-base("1A.F", base-from: 16, base-to: 8, explain: true, mode: "line")
```

1. Conversion from base 16 to base 10:

$$\overline{1A.F}^{16} = 1 \times 16^1 + 10 \times 16^0 + 15 \times 16^{-1} = \overline{26.9375}^{10}$$

2. Conversion from base 10 to base 8:

Integer part (successive divisions by 8) :

- $26 = 8 \times 3 + 2$
- $3 = 8 \times 0 + 3$

Fractional part (successive multiplications by 8) :

- $0.9375 \times 8 = 7.5$ (integer part: 7)
- $0.5 \times 8 = 4$ (integer part: 4)

Result: $\overline{1A.F}^{16} = \overline{32.74}^8$

Paramètres

```
base-to-base(  
  val: int | float | str,  
  base-from: int,  
  base-to: int,  
  explain: boolean,  
  equal: boolean,  
  precision: int,  
  fr: boolean,  
  mode: str  
) -> str | int | float
```

val int or float or str

nombre à convertir

base-from int

base depuis laquelle convertir

Par défaut: 2

base-to int

base vers laquelle convertir

Par défaut: 10

explain `boolean`

pour avoir le détail de la conversion

Par défaut: `false`

equal `boolean`

pour avoir l'égalité de la conversion

Par défaut: `false`

precision `int`

permet de limiter le nombre de chiffres après la virgule si le développement est infini

Par défaut: `6`

fr `boolean`

mode fr pour avoir des virgules à la place des points

Par défaut: `false`

mode `str`

différents modes de présentation « paren », « line », « simple »

Par défaut: `"paren"`

division

Division du décimal a par le second b

avec : `extradigits` chiffres en plus après la virgule que dans a (si `cycle:false`), ils apparaissent en gris (mode: « g » par défaut) ou blanc (mode: « 0 ») ou des points (mode: « p » ou « gp ») ou rien (mode: « »), `s:true/false` pour avoir ou non les soustractions, Possibilité de souligner en couleur (`cycle-color`) le cycle dans le quotient avec `cycle:true` ou de l'avoir entre parenthèses `cycle:""` ou en couleur `cycle:red` Possibilité de présenter des divisions à trous avec `hide-result`, `list` et `solution` (de paramètres `type-mask`, `border-color` et `solution-color`)

Exemples :

```
#division(12.6, 3.75) #h(1cm)
// En hexadécimal avec décimales au
quotient et des flèches montrant quand
les chiffres descendent :
#division(base: 16, extradigits: 2,
"1A8", "B", arrows: true)
```

```
// on peut avoir le cycle complet
automatiquement et cacher les
soustractions :
#division(121.5, 7, s:false,
cycle:true)
// en présentation anglo-saxonne :
#division(121.5, 7, cycle:true,
fr:false, dx:.5em, dy:-.9em)
```

Paramètres

```
division(
  a: int float,
  b: int float,
  base: int,
  show-base: boolean,
  mode: str,
  extradigits: int,
  s: boolean,
  cycle: boolean str color,
  cycle-color: color,
  fr: boolean,
  size: length relative length,
  dx,
  dy,
  hide-result: boolean,
  list: array,
  solution: boolean,
  type-mask: str,
  border-color: color,
  solution-color: color,
  space: length relative length fraction
) -> content
```

a int or float

dividende

b `int` or `float`

diviseur

base `int`

Base de numération (2 à 36)

Par défaut: `10`

show-base `boolean`

Montrer ou non dans quelle base l'addition est posée

Par défaut: `false`

mode `str`

Mode pour les zéros ajoutés au dividende : « g » (gris), « 0 » (noir), « p » (petit point), « gp » (grand point), « » (vide)

Par défaut: `"g"`

extradigits `int`

Nombre de décimales supplémentaires à calculer au quotient

Par défaut: `0`

s `boolean`

s: true/false pour afficher ou masquer les soustractions

Par défaut: `true`

cycle `boolean` or `str` or `color`

Détection et affichage automatique du cycle périodique

Par défaut: `false`

cycle-color `color`

couleur du cycle ou du soulignage

Par défaut: `luma(50%)`

fr `boolean`

Mode fr:true, le séparateur décimal est « , » sinon « . »

Par défaut: `true`

size `length` or `relative length`

Taille des colonnes du tableau

Par défaut: `.7em`

dx

dx for the) in mode fr:false

Par défaut: `.6em`

dy

dy for the) in mode fr:false

Par défaut: `-.82em`

hide-result `boolean`

Masquer les étapes et le quotient pour le mode exercice

Par défaut: `false`

list `array`

list des indices de cases à masquer (exercice à trous : a, b, étapes, quotient)

Par défaut: `()`

solution `boolean`

Afficher ou non la solution dans les masques

Par défaut: `false`

type-mask `str`

Type de masque (« rect » ou « line »)

Par défaut: `"rect"`

border-color `color`

Couleur de la bordure des cadres de masque

Par défaut: `blue.mix(gray)`

solution-color `color`

Couleur du texte de la solution

Par défaut: `red`

space `length` or `relative length` or `fraction`

Espacement si plusieurs sur la même ligne

Par défaut: `1fr`

egalite-decimale

Ecrit l'égalité d'une division décimale

```
#egalite-decimale(126, 25)
```

```
126 ÷ 25 = 5.04
```

Paramètres

```
egalite-decimale(  
  a: int float ,  
  b: int float ,  
  s: str  
) -> content
```

a `int` or `float`

dividende

b `int` or `float`

diviseur

s `str`

signe « e » pour =, sinon approx

Par défaut: `"e"`

egalite-euclidienne

Ecrit l'égalité d'une division euclidienne

```
#egalite-euclidienne(126, 27)
```

```
126 = 4 × 27 + 18
```

Paramètres

```
egalite-euclidienne(  
  a: int float,  
  b: int float  
) -> content
```

a int or float

dividende

b int or float

diviseur

from-base

conversion en décimal depuis une autre base (défaut : 2).

```
#from-base("1AF", base: 16,)
```

```
431
```

Paramètres

```
from-base(  
  val: int float str,  
  base: int,  
  fr: boolean,  
  show-base: boolean  
) -> str int float
```

val int or float or str

nombre à convertir

base int

base depuis laquelle convertir

Par défaut: 2

fr `boolean`

mode fr pour avoir des virgules à la place des points

Par défaut: `false`

show-base `boolean`

indique en indice que le résultat est en base 10

Par défaut: `false`

multiplication

Pose la multiplication de 2 nombres entiers ou décimaux avec possibilité de cacher les lignes avec `hide-result`, une list de certains chiffres puis d'afficher la solution, diverses couleurs paramétrables etc

Exemples :

```
#multiplication(12.5, 3.75) #h(1cm)
// Avec des chiffres cachés
#multiplication(1215, 142,list:
(3,12,19,36,40),type-mask: "line",)
```

$\begin{array}{r} 12,5 \\ \times 3,75 \\ \hline 625 \\ + 8750 \\ + 37500 \\ \hline = 46,875 \end{array}$	$\begin{array}{r} \dots 215 \\ \times 1\dots 2 \\ \hline 24\dots 0 \\ + 48600 \\ + 121500 \\ \hline 172\dots 30 \end{array}$
--	--

```
// En octal (base 8) avec des nombres
flottants et toutes les retenues :
#multiplication(base: 8, "7,4", 3.2,
carries: true, carries-color:
(red,blue)) #h(1cm)
// Uniquement les retenues de
multiplication (au-dessus du premier
terme) :
#multiplication(carries: "mult", 784,
53, carries-color: teal)
```

$\begin{array}{r} \overset{1}{7},4 \\ \times 3,2 \\ \hline 170 \\ + 2640 \\ \hline \overset{+1+1}{=} 30,30 \end{array}$	$\begin{array}{r} \overset{2}{7}\overset{1}{8}4 \\ \times 53 \\ \hline 2352 \\ + 39200 \\ \hline = 41552 \end{array}$
---	---

```
// En binaire avec uniquement les
retenues d'addition :
$#multiplication(base: 2, carries:
"add", "1011", "1101")$
```

$$\begin{array}{r} 1011 \\ \times 1101 \\ \hline 1011 \\ + 00 \\ + 101100 \\ + 1011000 \\ \hline \overset{+1+1+1+1}{=} 10001111 \end{array}$$

Paramètres

```
multiplication(  
  a: int float str,  
  b: int float str,  
  base: int,  
  show-base: boolean,  
  carries: boolean str,  
  carries-color: color array,  
  mode: str,  
  sym: boolean str array,  
  fr: boolean str,  
  size: length relative length,  
  hide-result: boolean,  
  list: array,  
  solution: boolean,  
  type-mask: str,  
  border-color: color,  
  solution-color: color,  
  space: length relative length fraction  
) -> content
```

a int or float or str

premier facteur

b int or float or str

second facteur

base int

Base de numération (2 à 36)

Par défaut: 10

show-base boolean

Montrer ou non dans quelle base l'addition est posée

Par défaut: false

carries boolean or str

Afficher les retenues : false, true (« all »), « mult », « add »

Par défaut: false

carries-color `color` or `array`

Couleur des retenues

Par défaut: `red`

mode `str`

Présentation des décalages : « g » (défaut) pour 0 en gris, « 0 » pour des 0 normaux, « p » pour des petits points et « gp » pour des points plus gros, « » ou autre pour rien

Par défaut: `"g"`

sym `boolean` or `str` or `array`

Afficher les + des additions et le égal du résultat si true, seulement les + si « + », seulement le = si « = », rien si false, sinon donner une list du sign (+, =) par ex

Par défaut: `true`

fr `boolean` or `str`

Mode fr:true, le séparateur décimal est « , » sinon « . »

Par défaut: `true`

size `length` or `relative length`

Largeur des colonnes

Par défaut: `1em`

hide-result `boolean`

Masquer les lignes intermédiaires et le résultat final pour les exercices

Par défaut: `false`

list `array`

list des indices de cases à masquer (pour multiplication à trous)

Par défaut: `()`

solution `boolean`

Afficher ou non la solution dans les masques

Par défaut: `false`

type-mask `str`

Type de masque (« rect » ou « line »)

Par défaut: "rect"

border-color `color`

Couleur de la bordure des cadres de masque

Par défaut: `blue.mix(gray)`

solution-color `color`

Couleur du texte de la solution

Par défaut: red

space `length` or `relative length` or `fraction`

Espacement si plusieurs sur la même ligne

Par défaut: `1fr`

racine

Calcul de la racine carrée de a, avec extradigits décimales en plus, en montrant les groups de 2 en groups-color, avec possibilité d'afficher les étapes avec step, de cacher les soustractions avec s, de cacher les résultats ou une list de chiffres

Exemples :

```
#racine(24368,size: 1em,list: (2,27))
```

$$\begin{array}{r|l} 24 \text{ } \textcolor{blue}{6} 8 & 1 \text{ } \textcolor{blue}{6} \\ \hline -1 & 1^2 = 1 < 2 \\ \hline -1 \text{ } 4 \text{ } 3 & 25 \times \textcolor{blue}{5} = 125 \\ \hline -1 \text{ } 2 \text{ } 5 & 306 \times \textcolor{blue}{6} = 1836 \\ \hline -1 \text{ } 8 \text{ } 6 \text{ } 8 & \\ \hline -1 \text{ } 8 \text{ } 3 \text{ } 6 & \\ \hline & 3 \text{ } 2 \end{array}$$

```
#grid(columns: 3*(1fr,), row-gutter: 1em,
  ..for i in range(1,11) {[
    ==== Étape #i
    #racine(24368, extradigits: 1,groups: true, step: i)
  ],)+if i == 1 {[[],[],)}]}
)
```

Étape 1

$$\begin{array}{r|l} \overline{24368} & 1 \\ -\frac{1}{1} & 1^2 = 1 < 2 \end{array}$$

Étape 2

$$\begin{array}{r|l} \overline{24368} & 1 \\ -\frac{1}{143} & 1^2 = 1 < 2 \\ & 2c \times c \leq 143 \end{array}$$

Étape 3

$$\begin{array}{r|l} \overline{24368} & 1 \\ -\frac{1}{143} & 1^2 = 1 < 2 \\ & 25 \times 5 = 125 \end{array}$$

Étape 4

$$\begin{array}{r|l} \overline{24368} & 15 \\ -\frac{1}{143} & 1^2 = 1 < 2 \\ -\frac{125}{18} & 25 \times 5 = 125 \end{array}$$

Étape 5

$$\begin{array}{r|l} \overline{24368} & 15 \\ -\frac{1}{143} & 1^2 = 1 < 2 \\ -\frac{125}{1868} & 25 \times 5 = 125 \\ & 30c \times c \leq 1868 \end{array}$$

Étape 6

$$\begin{array}{r|l} \overline{24368} & 15 \\ -\frac{1}{143} & 1^2 = 1 < 2 \\ -\frac{125}{1868} & 25 \times 5 = 125 \\ & 306 \times 6 = 1836 \end{array}$$

Étape 7

$$\begin{array}{r|l} \overline{24368} & 156 \\ -\frac{1}{143} & 1^2 = 1 < 2 \\ -\frac{125}{1868} & 25 \times 5 = 125 \\ -\frac{1836}{32} & 306 \times 6 = 1836 \end{array}$$

Étape 8

$$\begin{array}{r|l} \overline{24368,00} & 156 \\ -\frac{1}{143} & 1^2 = 1 < 2 \\ -\frac{125}{1868} & 25 \times 5 = 125 \\ -\frac{1836}{3200} & 306 \times 6 = 1836 \\ & 312c \times c \leq 3200 \end{array}$$

Étape 9

$$\begin{array}{r|l} \overline{24368,00} & 156 \\ -\frac{1}{143} & 1^2 = 1 < 2 \\ -\frac{125}{1868} & 25 \times 5 = 125 \\ -\frac{1836}{3200} & 306 \times 6 = 1836 \\ & 3121 \times 1 = 3121 \end{array}$$

Étape 10

$$\begin{array}{r|l} \overline{24368,00} & 156,1 \\ -\frac{1}{143} & 1^2 = 1 < 2 \\ -\frac{125}{1868} & 25 \times 5 = 125 \\ -\frac{1836}{3200} & 306 \times 6 = 1836 \\ -\frac{3121}{79} & 3121 \times 1 = 3121 \end{array}$$

Paramètres

```
racine(  
  a: int float ,  
  extradigits: int ,  
  groups: boolean ,  
  groups-color: color ,  
  step: none int ,  
  step-color: color ,  
  s: boolean ,  
  fr: boolean ,  
  size: length relative length ,  
  hide-result: boolean ,  
  list: array ,  
  solution: boolean ,  
  type: str ,  
  border-color: color ,  
  solution-color: color ,  
  space: length relative length fraction ,  
  ..args: arguments  
) -> content
```

a int or float

nombre

extradigits int

Nombre de tranches de « 00 » supplémentaires à calculer après la virgule

Par défaut: 0

groups boolean

Afficher des crochets au-dessus des tranches de 2 chiffres

Par défaut: false

groups-color color

Couleur des crochets

Par défaut: blue.darken(20%)

step none or int

Étape de résolution pas à pas

Par défaut: none

step-color `color`

couleur de l'étape

Par défaut: `blue.darken(20%)`

s `boolean`

Afficher ou masquer les soustractions sous le dividende

Par défaut: `true`

fr `boolean`

Mode fr:true, le séparateur décimal est « , » sinon « . »

Par défaut: `true`

size `length` or `relative length`

Largeur des colonnes

Par défaut: `0.65em`

hide-result `boolean`

Masquer les étapes, les opérations et le résultat (mode exercice)

Par défaut: `false`

list `array`

list des indices de cases à masquer (exercice à trous)

Par défaut: `()`

solution `boolean`

Afficher ou non la solution dans les masques

Par défaut: `false`

type `str`

Type de masque (« rect » ou « line »)

Par défaut: `"rect"`

border-color `color`

Couleur de la bordure des cadres de masque

Par défaut: `blue.mix(gray)`

solution-color `color`

Couleur du texte de la solution

Par défaut: `red`

space `length` or `relative length` or `fraction`

espacement à droite

Par défaut: `1fr`

..args `arguments`

autres arguments à la racine, ex: [...]

sous-en-ligne

Pour avoir la différence en ligne de tous les nombres saisis

```
$#sous-en-ligne(198.7,107.3,-35)$
```

```
198.7 - 107.3 - (-35) = 126.4
```

Paramètres

```
sous-en-ligne(  
  ..ops: arguments ,  
  digits: auto int  
) -> content
```

..ops `arguments`

termes de la soustraction

digits `auto` or `int`

nombre de chiffres de l'arrondi

Par défaut: `auto`

soustraction

Pose la soustraction de 2 ou plusieurs nombres avec possibilité de cacher la ligne de résultat avec `hide-result`, une list de certains chiffres puis d'afficher la solution, diverses couleurs paramétrables etc

Exemples :

```
#soustraction(12.5, 3.75)#h(1cm)
#soustraction(121.5, 3.75, .8,
14.2,list:(2,10,19))
```

$$\begin{array}{r} 1 \bigcirc 1, 50 \\ - \quad \bigcirc, 75 \\ - \quad 0, \bigcirc 0 \\ - \quad 14, 20 \\ \hline 102, 75 \end{array}$$

```
// En hexadécimal (Base 16) :
#soustraction(base: 16, "1A4.F",
"B2.C",show-base: true)
```

$$\begin{array}{r} 1A4, F \\ - \quad B2, C \\ \hline F2, 3 \end{array}^{16}$$

Paramètres

```
soustraction(
  base: int,
  show-base: boolean,
  show-borrow: boolean,
  canceled: boolean,
  fr: boolean,
  carries-color: color,
  size: length relative length,
  zeros: boolean,
  hide-result: boolean,
  border-color: color,
  solution-color: color,
  type-mask: str,
  list: array,
  solution: boolean,
  sign: boolean content,
  text-size: length relative length,
  space: length relative length fraction,
  ..args: array arguments
) -> content
```

base `int`

Base de calcul (de 2 à 36)

Par défaut: 10

show-base boolean

Montrer ou non dans quelle base l'addition est posée

Par défaut: false

show-borrow boolean

Montrer ou non les retenues

Par défaut: true

canceled boolean

Si fr:false, montrer ou non le nombre initial barré

Par défaut: false

fr boolean

Ecriture française ou avec emprunts à l'anglaise

Par défaut: true

carries-color color

couleur des retenues

Par défaut: red

size length or relative length

Taille des colonnes

Par défaut: 1em

zeros boolean

Ajoute ou non des zéros

Par défaut: true

hide-result boolean

Cache les retenues et le résultat

Par défaut: false

border-color `color`

Couleur du cadre des chiffres cachés

Par défaut: `blue.mix(gray)`

solution-color `color`

Couleur des chiffres cachés si `solution = true`

Par défaut: `red`

type-mask `str`

Comment les chiffres sont cachés : soient « rect » pour un rectangle arrondi soit ...

Par défaut: `"rect"`

list `array`

list des chiffres à cacher dans une addition à trous

Par défaut: `()`

solution `boolean`

Montrer les retenues et le résultat en mode `hide-result` (pour les corrections)

Par défaut: `false`

sign `boolean` or `content`

sign d'égalité, `false` ou `true` [=] ou un symbole \$ = \$

Par défaut: `false`

text-size `length` or `relative length`

taille du des chiffres : `1em`

Par défaut: `1em`

space `length` or `relative length` or `fraction`

Espacement si plusieurs sur la même ligne

Par défaut: `1fr`

..args `array` or `arguments`

termes de la soustraction

to-base

conversion d'un nombre en écriture décimale vers une autre base (défaut : 2).

```
#to-base(base:16,"7,8",fr:false)
#h(1cm)
#to-base(100, base: 8)
```

7.CCCCCC 144

Paramètres

```
to-base(
  val: int float str,
  base: int,
  fr: boolean,
  show-base: boolean,
  precision: int
) -> str int float
```

val int or float or str

nombre à convertir

base int

base vers laquelle convertir

Par défaut: 2

fr boolean

mode fr pour avoir des virgules à la place des points

Par défaut: false

show-base boolean

indique en indice que le résultat est en base 10

Par défaut: false

precision int

permet de limiter le nombre de chiffres après la virgule si le développement est infini

Par défaut: 6

Additions et soustractions de durées/horaires

- `addition-durees()`
- `prepa-duree()`
- `soustraction-durees()`

addition-durees

Pose l'addition de deux ou plusieurs durées.

Chaque durée est un tableau de l'une des formes suivantes :

- (jours, heures, minutes, secondes).
- (heures, minutes, secondes).
- (minutes, secondes).

Exemples :

```
#addition-durees(  
  (1, 2, 45, 50),  
  (0, 1, 20, 30),  
)
```

j	h	min	s
	1	1	
1	02	45	50
+	0	01	20
	1	04	06
			20

```
#addition-durees(  
  (  
    (1, 2, 45, 50),  
    (0, 1, 20, 30),  
    (2, 23, 55, 45),  
  ),  
  hide-result: true,  
)
```

j	h	min	s
1	2	2	
1	02	45	50
+	0	01	20
+	2	23	55
			45

Paramètres

```
addition-durees(  
  show-carry: boolean ,  
  carries-color: color ,  
  carry-size: length relative length ,  
  size: length relative length ,  
  conversion: boolean ,  
  hide-empty: boolean ,  
  hide-result: boolean ,  
  border-color: color ,  
  solution-color: color ,  
  type-mask: str ,  
  list: array ,  
  solution: boolean ,  
  sign: boolean content ,  
  text-size: length relative length ,  
  show-units: boolean ,  
  units: array ,  
  units-size: length relative length ,  
  units-color: color ,  
  zero-pad: boolean ,  
  align: boolean ,  
  verticals: boolean ,  
  ..args: array arguments  
) -> content
```

show-carry boolean

Afficher les retenues.

Par défaut: **true**

carries-color color

Couleur des retenues.

Par défaut: red

carry-size length or relative length

Taille des retenues et des symboles + et =, max 1em.

Par défaut: **.8em**

size length or relative length

Largeur des colonnes correspondant aux unités.

Par défaut: **2.5em**

conversion `boolean`

Mode conversion du résultat à la place des retenues.

Par défaut: `false`

hide-empty `boolean`

Cacher les colonnes n'ayant que des zéros

Par défaut: `true`

hide-result `boolean`

Masquer le résultat.

Par défaut: `false`

border-color `color`

Couleur du cadre des valeurs masquées.

Par défaut: `blue.mix(gray)`

solution-color `color`

Couleur des valeurs dans la correction.

Par défaut: `red`

type-mask `str`

Type de masque : « rect » ou « line ».

Par défaut: `"rect"`

list `array`

Indices des cellules à masquer.

Par défaut: `()`

solution `boolean`

Afficher la solution des cellules masquées.

Par défaut: `false`

sign `boolean` or `content`

sign placé devant le résultat.

Par défaut: `false`

text-size `length` or `relative length`

Taille du texte.

Par défaut: `1em`

show-units `boolean`

Afficher les noms des unités.

Par défaut: `true`

units `array`

Noms des quatre unités.

Par défaut: `([j], [h], [min], [s])`

units-size `length` or `relative length`

Taille des unités, max 1em

Par défaut: `.75em`

units-color `color`

couleur des unités

Par défaut: `gray`

zero-pad `boolean`

Afficher heures, minutes et secondes sur deux chiffres.

Par défaut: `true`

align `boolean`

Aligner les chiffres des unités.

Par défaut: `true`

verticals `boolean`

Pointillés verticaux pour séparer les colonnes

Par défaut: `false`

..args `array` or `arguments`

liste des durées

prepa-duree

Convertis des durées décimales ou fractionnaires en durées entières

Paramètres

`prepa-duree(..args: array arguments) -> array`

..args `array` or `arguments`

list de 2 à 4 nombres (j,h,min,s) à (min, s)

soustraction-durees

Pose la soustraction de deux durées.

Chaque durée est un tableau de l'une des formes suivantes :

- (jours, heures, minutes, secondes) ;
- (heures, minutes, secondes) ;
- (minutes, secondes).

La première durée doit être supérieure ou égale à la seconde.

Exemples :

```
#soustraction-durees(  
  (2, 4, 15, 10),  
  (1, 6, 45, 30),  
)
```

	j	h	min	s
			⁺²⁴	⁺⁶⁰
	2	04	15	10
-	1	06	45	30
	⁺¹	⁺¹	⁺¹	
	0	21	29	40

```
#soustraction-durees(  
  (2, 15, 10),  
  (1, 45, 30),  
  list:(7,10),  
)
```

	h	min	s
	2	15	10
-	1	45	
	0		40

Paramètres

```
soustraction-durees(  
  show-borrow: boolean ,  
  carries-color: color ,  
  carry-size: length relative length ,  
  size: length relative length ,  
  conversion: boolean ,  
  hide-empty: boolean ,  
  hide-result: boolean ,  
  border-color: color ,  
  solution-color: color ,  
  type-mask: str ,  
  list: array ,  
  solution: boolean ,  
  sign: boolean content ,  
  text-size: length relative length ,  
  show-units: boolean ,  
  units: array ,  
  units-size: length relative length ,  
  units-color: color ,  
  zero-pad: boolean ,  
  align: boolean ,  
  verticals: boolean ,  
  ..args: array arguments  
) -> content
```

show-borrow boolean

Afficher les emprunts et les retenues.

Par défaut: **true**

carries-color color

Couleur des emprunts et retenues.

Par défaut: red

carry-size length or relative length

Taille des retenues et des symboles + et =, max 1em.

Par défaut: **.7em**

size length or relative length

Largeur des colonnes.

Par défaut: **2.5em**

conversion `boolean`

Mode conversion du minuende à la place des retenues

Par défaut: `false`

hide-empty `boolean`

Cacher les colonnes n'ayant que des zéros

Par défaut: `true`

hide-result `boolean`

Masquer le résultat.

Par défaut: `false`

border-color `color`

Couleur du cadre des valeurs masquées.

Par défaut: `blue.mix(gray)`

solution-color `color`

Couleur des valeurs dans la correction.

Par défaut: `red`

type-mask `str`

Type de masque : « rect » ou « line ».

Par défaut: `"rect"`

list `array`

Indices des cellules à masquer.

Par défaut: `()`

solution `boolean`

Afficher la solution.

Par défaut: `false`

sign `boolean` or `content`

sign placé devant le résultat.

Par défaut: `false`

text-size `length` or `relative length`

Taille du texte.

Par défaut: `1em`

show-units `boolean`

Afficher les noms des unités.

Par défaut: `true`

units `array`

Noms des quatre unités.

Par défaut: `([j], [h], [min], [s])`

units-size `length` or `relative length`

Taille des unités, max 1em.

Par défaut: `.75em`

units-color `color`

couleur des unités.

Par défaut: `gray`

zero-pad `boolean`

Afficher heures, minutes et secondes sur deux chiffres.

Par défaut: `true`

align `boolean`

Aligner les chiffres des unités.

Par défaut: `true`

verticals `boolean`

Pointillés verticaux pour séparer les colonnes

Par défaut: `false`

..args `array` or `arguments`

liste des durées

Calculs détaillés et mise en avant

- `detail()`
- `etapes-calcul()`

detail

Fonction pour détailler les calculs manuellement

```
#detail("A=15/21=(5*r3)/  
(7*r3)=5/7",add-  
rounded:true,c:"circle")
```

$$A = \frac{15}{21} = \frac{5 \times \textcircled{3}}{7 \times \textcircled{3}} = \frac{5}{7} \approx 0,71$$

```
#detail("B=(15s2)/(21s18)=(5*r3 s2)/  
(7*r3 s(9*2))=(5 ps2)/(21 bs2)  
=5/21",c:"color",vertical:true, add-  
rounded: true)
```

$$\begin{aligned} B &= \frac{15\sqrt{2}}{21\sqrt{18}} \\ &= \frac{5 \times \textcolor{red}{3}\sqrt{2}}{7 \times \textcolor{red}{3}\sqrt{9 \times 2}} \\ &= \frac{5\sqrt{2}}{21\sqrt{2}} \\ &= \frac{5}{21} \\ &\approx 0,24 \end{aligned}$$

```
#detail("cal(A)=pi * r^2 = pi*5^2=25  
pi",add-rounded: true,digits: 4)
```

$$\mathcal{A} = \pi \times r^2 = \pi \times 5^2 = 25\pi \approx 25 \times 3,1416 \approx 78,54 \approx 78,54$$

Paramètres

```
detail(  
  expr-str: str,  
  vertical: boolean,  
  add-rounded: boolean,  
  digits: int,  
  appro: int,  
  todo: boolean,  
  space: length relative length fraction,  
  a: color,  
  b: color,  
  f: color,  
  g: color,  
  r: color,  
  l: color,  
  m: color,  
  n: color,  
  o: color,  
  t: color,  
  p: color,  
  y: color,  
  c: str  
) -> content
```

expr-str `str`

Le calcul détaillé

vertical `boolean`

mode vertical ou non

Par défaut: `false`

add-rounded `boolean`

Si ajout d'un arrondi à la fin ou non

Par défaut: `false`

digits `int`

le nombre de chiffres de l'arrondi

Par défaut: `2`

appro `int`

rang à partir duquel remplacer les = par des $\approx$ si besoin

Par défaut: `0`

todo `boolean`

si todo:true, seul l'expression initiale est affichée

Par défaut: `false`

space `length` or `relative length` or `fraction`

l'espace de séparation avant le calcul suivant

Par défaut: `1fr`

a `color`

a:aqua peut être changée de la couleur typst par défaut si besoin

Par défaut: aqua

b `color`

b: blue peut être changée de la couleur typst par défaut si besoin

Par défaut: blue

f `color`

f: fuchsia peut être changée de la couleur typst par défaut si besoin

Par défaut: fuchsia

g `color`

g: green peut être changée de la couleur typst par défaut si besoin

Par défaut: green

r `color`

r: red peut être changée de la couleur typst par défaut si besoin

Par défaut: red

l `color`

l: gray peut être changée de la couleur typst par défaut si besoin

Par défaut: gray

m `color`

m: maroon peut être changée de la couleur typst par défaut si besoin

Par défaut: maroon

n `color`

n: `rgb(8, 111, 189)` -> navy peut être changée si besoin

Par défaut: `rgb("#086fbd")`

o `color`

o: orange peut être changée de la couleur typst par défaut si besoin

Par défaut: orange

t `color`

t: `rgb(0,128,128)` -> teal peut être changée si besoin

Par défaut: `rgb(0,128,128)`

p color

p: purple peut être changée de la couleur typst par défaut si besoin

Par défaut: purple

y color

y: yellow peut être changée de la couleur typst par défaut si besoin

Par défaut: yellow

c str

choix du mode « cancel » ou « circle » ou « color »

Par défaut: "color"

etapes-calcul

Fait le calcul détaillé de expr-str (sauf si todo:true) avec possibilité d'avoir les calculs indépendants de même priorité fait simultanément avec concomitant, en mode vertical ou horizontal, et mise en avant de l'étape en cours (highlight) avec un mode fraction forçant la poursuite des calculs avec des fractions

```
#etapes-calcul("[ 6 + 5 * (23 + 8 /  
2) ] * 8", fraction: false, vertical:  
true,n:0)
```

$$\begin{aligned} & \left[6 + 5 \times \left(23 + \frac{8}{2} \right) \right] \times 8 \\ &= [6 + 5 \times (23 + 4)] \times 8 \\ &= [6 + 5 \times 27] \times 8 \\ &= [6 + 135] \times 8 \\ &= 141 \times 8 \\ &= 1128 \end{aligned}$$

```
#etapes-calcul("(200 - 45 * 2) / ((4 *  
7 - 3 : 6) * 2)",concomitant: true)
```

$$\frac{200 - 45 \times 2}{(4 \times 7 - 3 \div 6) \times 2} = \frac{200 - 90}{(28 - 0,5) \times 2} = \frac{110}{27,5 \times 2} = \frac{110}{55} = 2$$

```
#etapes-calcul("2 * pi * 3", fraction:  
false, name: "B",n:3)
```

$$B = 2 \times \pi \times 3 = 2\pi \times 3 = 6\pi \approx 18,85$$

```
#etapes-calcul("ln(exp(1)) + 3",  
fraction: true, name: "E")
```

$$E = \ln(\exp(1)) + 3 = 1 + 3 = 4$$

```
#etapes-calcul("sqrt(8)*sqrt(27)",  
fraction: true)
```

$$\sqrt{8} \times \sqrt{27} = 2\sqrt{2} \times \sqrt{27} = 2\sqrt{2} \times 3\sqrt{3} = 6\sqrt{6}$$

Paramètres

```
etapes-calcul(  
  expr-str: str,  
  mode: str,  
  concomitant: boolean,  
  vertical: boolean,  
  digits: int,  
  n: int,  
  name: str content,  
  todo: boolean,  
  space: length relative length fraction,  
  fraction: boolean,  
  highlight: color boolean none,  
  skip: array  
) -> content
```

expr-str str

l'expression à calculer

mode str

mode « fr » ou « en » pour le séparateur décimal

Par défaut: "fr"

concomitant boolean

Passer à true pour avoir les calculs indépendants de même priorité fait simultanément

Par défaut: false

vertical boolean

Pour avoir une présentation « en colonne » du calcul

Par défaut: false

digits int

Nombre de décimales dans les calculs avec pi ...

Par défaut: 2

n `int`

Nombre d'étapes (comptées depuis la fin) auquel remplacer le = par un approx si besoin

Par défaut: `0`

name `str` or `content`

Nom à ajouter au début de l'expression

Par défaut: `" "`

todo `boolean`

Mode énoncé, seul l'expression proposé + un = apparaît

Par défaut: `false`

space `length` or `relative length` or `fraction`

Espacement horizontal entre deux etapes-calcul

Par défaut: `1fr`

fraction `boolean`

En mode fraction, le calcul reste exact : soit en mode fraction soit en s'arrêtant à 5pi par ex.

Par défaut: `false`

highlight `color` or `boolean` or `none`

Soit une couleur qui met en avant les calculs en cours soit false pour du gras ou none pour rien du tout

Par défaut: `rgb( "#1e88e5" )`

skip `array`

list des étapes à sauter si nécessaire

Par défaut: `()`